

The future of glitching

Tuesday, October 6, 2026

I purchased a Matrix Coolrunner Rev D back in 2017. This little chip is called a glitcher and it's a type of modchip. It's sole purpose, the reason it was designed, is to deliver a short pulse of energy (~ 100 nanoseconds) to an Xbox 360 Console.



You might ask yourself, "What's the point in that?". The answer is quite interesting.

Microsoft are good at security

It boils down to the simple fact that Microsoft learned from the mistakes of their previous console (the original xbox) and designed security well on their successor.

So well, that it was deemed essentially unhackable by software based approaches:

tmblnc said it himself, software based approaches of running unsigned code on the 360 mostly don't work, it was designed to be secure from a software point of view.¹

The entry point for the system, dubbed 1BL, is physically baked into the silicon of the Xbox 360's CPU². The job of 1BL is to load the next phase of the bootloader, called CB from flash storage, and verify that it was signed by microsoft.

So without Microsoft's signing keys, there was no real way to hijack control of the system during the boot process. And it only becomes harder as the console boots up through its different phases. Additional security features are turned on as it cascades up the boot chain. And they would all need individually circumvented to get any code running on the console that Microsoft haven't been over with a fine toothed comb.

So if the door to the castle is locked, and the guards, who are wise to equine ploys, are checking ID at the door, how does one get in? Well you don't use a key. You inform the guard that his shoelaces are untied and clobber him over the head with a club.

Finding the weak link

The point is that if the lock itself is secure, and the procedures surrounding entry are sound, sometimes the weakest link becomes something more fundamental. In our metaphor this is a person, but in the case of the xbox 360, it is the fact that computation relies on stable energy conditions to behave in a predicatable manner.

And this is the idea that glitching subverts. In the case of the Xbox 360, researchers found that by very quickly toggling a reset signal on the CPU (CPU_RESET), a signal that is usually supposed to reset the CPU, something far much more interesting happened. Instead of resetting, the CPU executed the current instruction incorrectly.

This gave the researchers a very useful primitive. They found that an instruction integral to the signature verification process, the mr (move register) instruction, could be glitched into ignoring a value it was about to move and instead move a 0 into its destination register.

So by simply knowing when, and toggling the lights on and off, they gained arbitrary code execution.

The future of glitching

It is astounding to me that this kind of esoteric side channel exists at all. But I think the larger point is that it's becoming more common.

The sort of trust chains that made the Xbox 360 hard to hack 10 years ago have now become common place in the world of computing hardware. Microsoft themselves began enforcing hardware backed secure boot for their Operating Systems back in 2021 x

More recently in 2024, the security features of an ESP32 v3 (secure boot and flash encrpytion) were bypassed using a fault injection attack y.

And there are recent examples specifically leveraging the Raspberry Pi Pico Microcontroller to perform similar fault injection attacks:

- Apple airtags being hacked within 2 weeks of their release in 2021 by stacksmashing using a Raspberry Pi Pico to fault inject the boot process
- The PicoFly modchip , released in 2023 for the Nintendo Switch console, using fault injection to defeat secure boot

To me, this suggests that as fault injection has become a more attractive attack vector (due to a hardened security ecosystem), it has in turn also become more accessible than ever in recent years, with the advent of cheap and accurate microcontrollers such as the Pi Pico.

A love letter to the Pico

The pico itself is versatile because it allows for very precise timing via its PIO blocks. These specialised blocks come with enough primitives to implement many standard protocols and have an effective timing resolution of ~7.5ns due to the 133 Mhz clock (

$$\frac{1}{133\text{ MHz}} \approx 7.52\text{ns} \text{).}$$

I have seen this versatility myself first hand when I finally dusted off the Matrix Coolrunner in 2026 and decided to take another crack at modchipping my xbox 360.

Back in 2017, when I bought the thing, it was quite expensive to modchip an Xbox 360 (for a broke student). You needed a number of tools to successfully mod one:

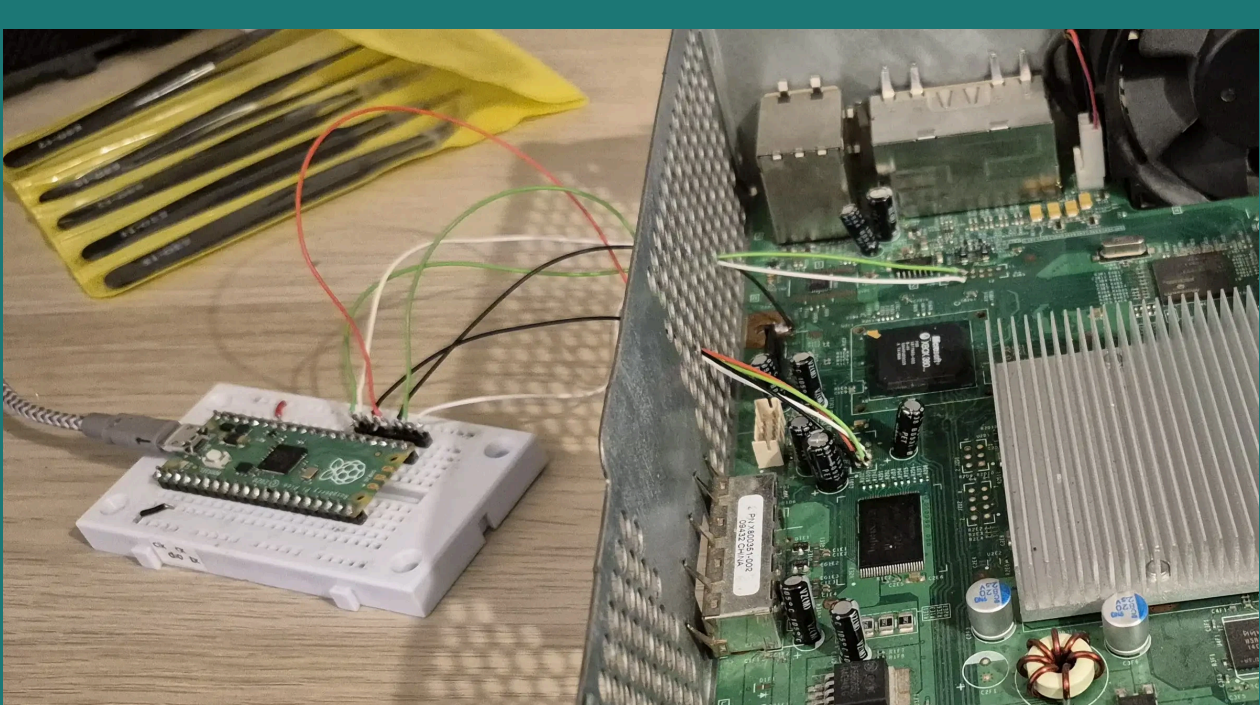
- A glitching chip
- Solder and an iron
- A NAND programmer
- A JTAG programmer

I tried to save some money by building my own NAND / JTAG programmer using the printer port (LPT port) on a very old PC that I had in reserve for the specific purpose. If I had ever got this working it would have taken over 30 minutes to dump the NAND. I no longer remember why, but I eventually gave up and shelved the project.

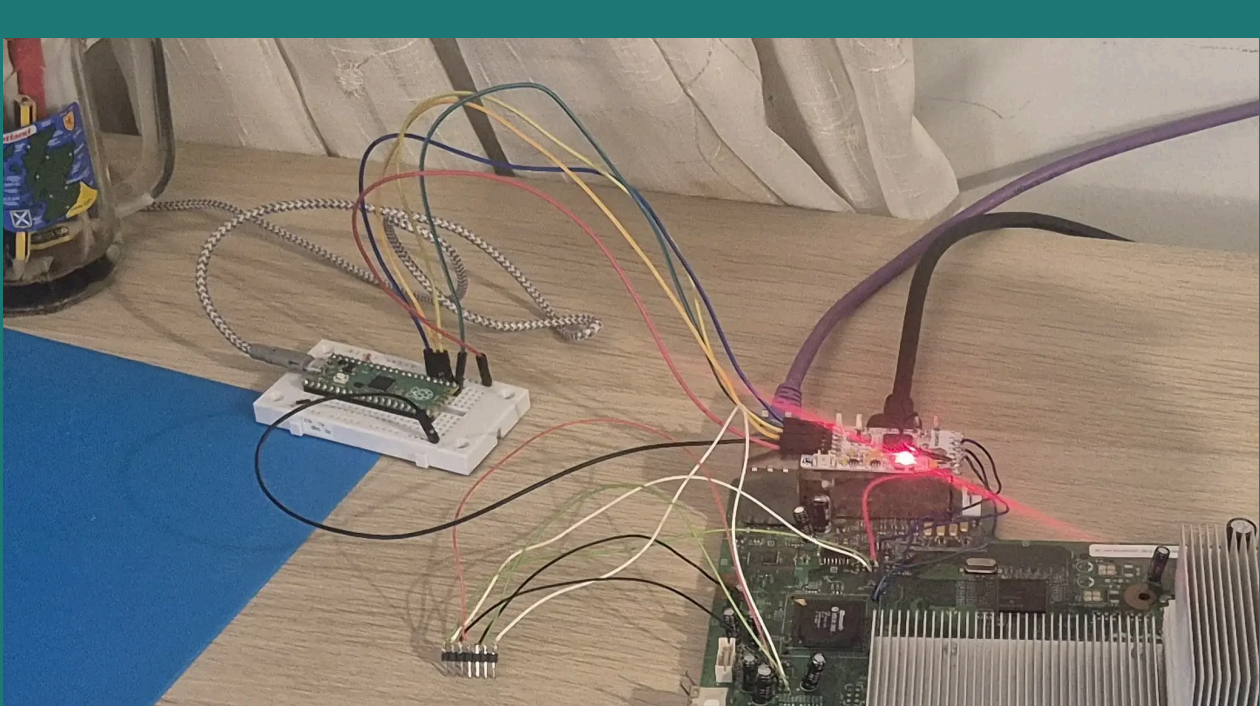


Line Print Terminal (LPT) Wiring Diagram: Superben51, Instructables

Reattempting this same procedure in 2026 was a breeze by comparison. I was able to flash PicoFlasher onto a Pi Pico and solder it to the correct pads on my Xbox 360. The dump operation then only took a handful of seconds to run.



Similarly I flashed Pico-dirtyJTAG onto a Pi Pico and was able to program the coolrunners timing files using Octal450's Timing Files and installing the urjtag project from source.



Summary

In summary, the future seems bright for

References

1. GliGli and Tiros, Reset Glitch Hack, accessed 7 October 2026. https://github.com/gligli/tools/blob/master/reset_glitch_hack/reset_glitch_hack.txt ↔
2. Free60Project, Boot_Process, Free60, accessed 7 October 2026. https://free60.org/System-Software/Boot_Process/ ↔